Inteligência Artificial

Inês Correia

2017/18

"Inteligência"

- capacidade de adquirir e aplicar conhecimento
- capacidade de pensar e raciocinar
- conj. de capacidades superiores da mente

pensar como o ser humano	pensar racionalmente
agir como o ser humano	agir racionalmente

Agentes ← entidade c/ poder/autoridade de agir
" ← q atua em representação de outrem

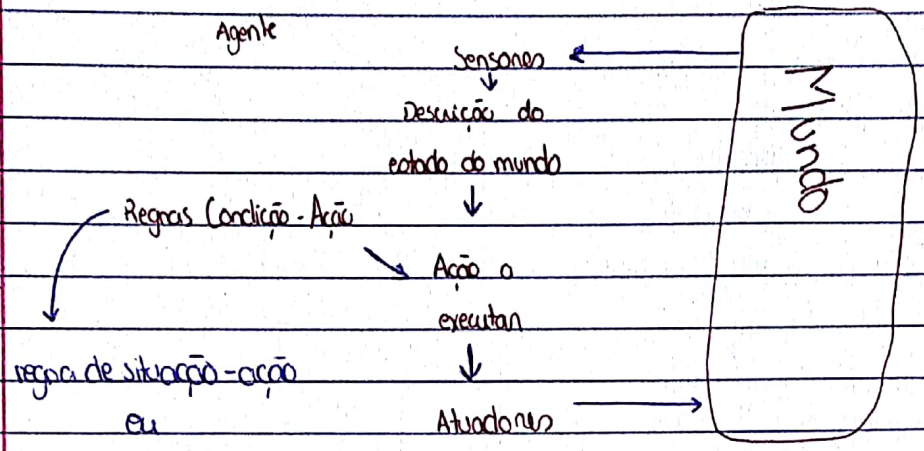
entidade c/ capacidade de obter informação sobre o seu ambiente e de executar ações em função dessa informação

→ Tipos de agentes
- reativos simples
- reativos c/ estados
- deliberativos orientados por objetivos
- " " " funções de utilidades

→ Arquiteturas
- Subsunção
- 3 torres
- 3 camadas
- CARL

Scanned by CamScanner

Agente reativo : simples



Sistemas de Quadro Preto

- elaboração dos sistemas reativos cl estado interno
- F.C. - "fonte de conhecimento" — altera o quadro preto

↓

cada uma rege-se por um conj. de regras de situação - ação

Propriedades do mundo de um agente

- Acessibilidade : sensores do agente permitem obter uma descrição completa do estado do mundo; é possível obter toda a informação relevante no processo de escolha das ações.
- Determinismo: estado resultante da execução de uma acção é totalmente determinado pelo estado atual e pelos efeitos esperados da acção.
- Mundo episódico: no caso em q cada episódio de percepção - acção é totalmente independente dos outros.
- Dinamismo: estado pode mudar enquanto o agente delibera ; caso contrário, é estático.
- Continuidade: evolução do estado do mundo é um processo contínuo (sem saltos) ; caso contrário, é discreto.

Scanned by CamScanner

Arquiteturas de agentes: Subsunção

↳ procura estabelecer a ligação entre percepção e ação a vários níveis

↓↓

organização em camadas → camada ⊕ baixa é a ⊕ reativa

• O peso da componente deliberativa aumenta à medida que se sobe na estrutura de camadas

Redes semânticas

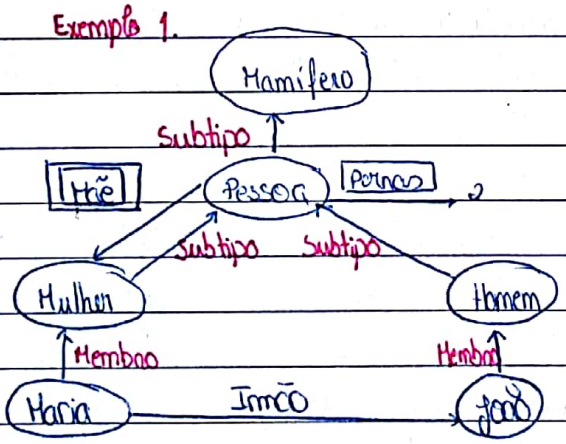
↓↳ representações gráficas do conhecimento

vantagem: legibilidade

→ Tipos de relações ← (confirmar!)

- Sub-tipo: $A \subset B$ (A contido em B) > permitem a herança de propriedades *
- Membro: $A \in B$ (instância)
- Relação objeto-objeto: $R(A,B)$
- Relação conjunto-objeto: $\forall x, x \in A \Rightarrow R(x,A)$
- Relação conjunto-conjunto: $\forall x, x \in A \Rightarrow \exists y (y \in B \wedge R(x,y))$

Exemplo 1.



Herança

* sub-tipo herda todas as propriedades do tipos ⊕ abstratos dos quais descende; a instância herda todas as propriedades do tipo a q pertence

Inferência: seguimento das ligações entre entidades cl vista à herança de propriedades

→ implementação de raciocínio não monotónico através do estabelecimento de valores por defeito e o correspondente cancelamento da herança.

Scanned by CamScanner

Métodos e Demónios

- Métodos: semântica similar à da programação orientada por objetos
- Demónios: procedimentos cuja execução é disparada automaticamente após certas operações de leitura ou escrita são efetuadas

UML / Diagramas de Classes

unified modelling language

↳ definem as relações existentes entre os diferentes classes de objetos num dado domínio

→ Classe: descrição de um conj. de objetos q partilham os mesmos atributos, operações, relações e semântica

objetos: físicos; conceitos q n tenham uma existência palpável

→ Atributo: propriedade de uma classe

→ Operação / método: implementação de um serviço q pode ser executado por qq objeto da classe

→ Instância de uma classe: objeto pertencente a essa classe

↓

"herdam" atributos e métodos da sua classe

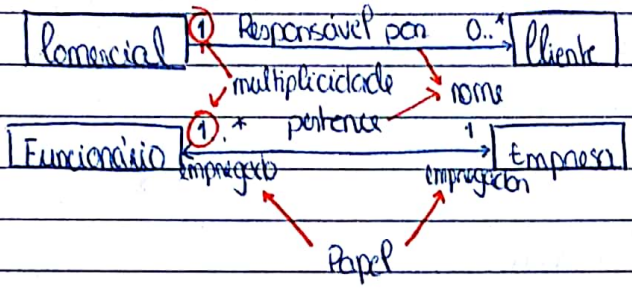
Relações

- Associação: classe A uma a interage com classe B
  ex.: aluno lê um livro
- Composição: relação todo-parte
  ex: um recibo tem vários itens
- Agregação: relação estrutural entre instâncias de 2 classes, em q a classe agregada existe independentemente da outra
  ex.: uma biblioteca tem vários livros

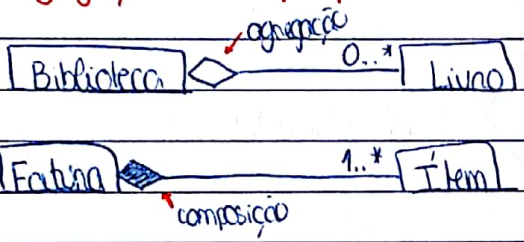
Scanned by CamScanner

- Generalização: classe A generaliza classe B e B especializa A
  ex.: um aluno é uma pessoa

Associação: Exemplos



Agregação e Composição



Redes Semânticas vs UML

Redes Semânticas	UML	
subtipo(subtipo, Tipo)	Generalização em diagramas de classes	
membro(obj, Tipo)	diagramas de objetos	
Relação Obj-Obj	associação, agregação e composição	→ diagramas objetos
" Obj-Tipo	não tem	
" Tipo-Tipo	associação, agregação e composição	→ diagramas classes

Indução vs Dedução

Indução: permite inferir casos gerais a partir de particulares

Dedução: preserva a verdade; permite inferir casos particulares a partir de regras gerais

Scanned by CamScanner

Indução : exemplo

Casos
O gato Tareco gosta de leite
O gato Pirata gosta de leite

⇒ Regra inferida
os gatos (normalmente) gostam de leite

"herança de baixo para cima"

Redes Semânticas em Python

⇒ Conj. de declarações

↓

cada uma associa uma relação semântica ao indivíduo q a declara

Declaration (User, Relation)

3 tipos:

Operações principais:
- insert - introduzir nova declaração
- query_local - questionar a rede semântica sobre as declarações existentes

Member(Obj, Type) → objeto é membro de um tipo

Subtype(Subtype, SuperType) → um tipo é subtipo de outro

Association(Ent1, AssocName, Ent2)

uma entidade está associada a outra

↓

obj / tipo

Scanned by CamScanner

Lógicas

Sintaxe	Semântica	Regras de inferência
conj. de frases / fórmulas q é possível escrever ⇒ WFF	relação entre as frases escritas nessa linguagem e os factos q a representam	permitem manipular as frases, gerando umas a partir das outras; base do processo de raciocínio

Lógica Proposicional

⇓

baseada em proposições

⇓

frase declarativa elementar q pode ser verdadeira / falsa — Ex.: "a neve é branca"; "o açúcar é um hidrocarboneto"

→ Variável proposicional: variável q toma o valor de verdade de uma dada proposição

Lógica de primeira ordem

Componentes:

- objetos / entidades
- expressões funcionais
- predicados / relações (frases elementares)

Conectivas lógicas (combina frases lógicas elementares para obter frases ⊕ complexas)

- ∧ (conjunção)
- ∨ (disjunção)
- ⇒ (implicação)
- ¬ (negação)

Quantificadores

Universal

- $\forall x\ A \equiv$ "qqr q seja x, a fórmula A é verdadeira"
- se A é uma fórmula bem formada, então $\forall x A$ tbm é.

Existencial

- $\exists x\ A =$ "Existe um x, para o qual a fórmula A é verdade"
- se A é uma fórmula bem formada, então $\exists x A$ tbm é

Scanned by CamScanner

Exemplos

- "Todos em Oxford são espertos"
  $\forall x$ Estuda$(x, Oxford) \Rightarrow$ Esperto(x)
  Erro comum: $\forall x$ Estuda$(x, Oxford) \wedge$ Esperto(x)
  $\simeq$ "Todos estão em Oxford e todos são espertos"

- "Algum em Oxford é esperto"
  $\exists x$ Estuda$(x, Oxford) \wedge$ Esperto(x)
  Erro comum: $\exists x$ Estuda$(x, Oxford) \Rightarrow$ Esperto(x)

- "Existe uma pessoa q gosta de toda a gente"
  $\exists x \forall y$ Gosta(x, y)

Lógica - Regras de Substituição

- Leis de DeMorgan
  $\neg(A \wedge B) \equiv \neg A \vee \neg B$
  $\neg(A \vee B) \equiv \neg A \wedge \neg B$

- Dupla negação
  $\neg\neg A \equiv A$

- Definição da implicação:
  $A \Rightarrow B \equiv \neg A \vee B$

- Transposição
  $A \Rightarrow B \equiv \neg B \Rightarrow \neg A$

- Comutação:
  $A \wedge B \equiv B \wedge A$
  $A \vee B \equiv B \vee A$

- Associação
  $(A \wedge B) \wedge C \equiv A \wedge (B \wedge C)$
  $(A \vee B) \vee C \equiv A \vee (B \vee C)$

- Distribuição
  $A \wedge (B \vee C) \equiv (A \wedge B) \vee (A \wedge C)$
  $A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$

- Leis de DeMorgan para lógica de 1ª ordem
  $\neg(\forall x\, P(x)) \equiv \exists x \neg P(x)$
  $\neg(\exists x\, P(x)) \equiv \forall x \neg P(x)$

Scanned by CamScanner

CNF e Forma Clausal

Forma normal conjuntiva → conj. de cláusulas → disjunção de literais → fórmula atómica*/ negação

* na lógica proposicional é uma proposição

Forma clausal → representação de CNF através do conj. das respetivas cláusulas

Conversão de uma fórmula proposicional para CNF e forma clausal

Passos:
- remover implicações
- reduzir o âmbito de aplicação das negações
- associar e distribuir até obter a forma CNF

Ex.:

Fórmula original: $A \Rightarrow (B \wedge C)$ ** $A \Rightarrow B \equiv \neg A \vee B$ **

Após remoção de implicações: $\neg A \vee (B \wedge C)$

Forma CNF: $(\neg A \vee B) \wedge (\neg A \vee C)$

Forma clausal: $\{\neg A \vee B, \neg A \vee C\}$

Conversão para forma clausal em lógica de primeira ordem

- renomear variáveis, de forma a q cada quantificador tenha 1a variável ≠
- remover as implicações
- reduzir o âmbito das negações (aplicar a negação)
- aplicar regras de substituição
- Skolemização (eliminação de quant. existenciais); • remover quant. universais *

Ex.: * continua a seguir

Fórmula original: $\forall x \forall y \neg (p(x,y) \Rightarrow \forall y\, q(y,y))$

Variáveis renomeadas: $\forall a \forall b \neg (p(a,b) \Rightarrow \forall c\, q(c,c))$

Implicações removidas: $\forall a \forall b \neg (\neg p(a,b) \vee \forall c\, q(c,c))$

Negações aplicadas: $\forall a \forall b (p(a,b) \wedge \exists c \neg q(c,c))$

Skolemizada aplicada: $\forall a \forall b (p(a,b) \wedge \neg q(f(a,b), f(a,b)))$

Quantificadores removidos: $p(a,b) \wedge \neg q(f(a,b), f(a,b))$

Scanned by CamScanner

- Converter para CNF
- Converter para a forma clausal (eliminar conjunções)
- Renomear variáveis

Forma clausal: $\{p(a,b), \neg q(f(a,b), f(a,b))\}$
Variáveis renomeadas: $\{p(a_1,b_1), \neg q(f(a_2,b_2), f(a_2,b_2))\}$

Consequência lógica: A é consequência lógica do conj. de fórmulas em Δ ($\Delta \models A$), se A toma valor "verdadeiro" em todas as interpretações para as quais cada uma das fórmulas em Δ toma tbm o valor verdadeiro

Prova: uma seq. de fórmulas $\{A_1, A_2, \ldots, A_n\}$ é uma prova (dedução) de A_n a partir de um conj. de fórmulas Δ se e só se cada uma das fórmulas A_i está em Δ ou pode ser inferida a partir das fórmulas $A_1 \ldots A_{i-1}$.
$\Delta \vdash A_n$

Correção: todas as fórmulas geradas pelo conj de regras de inferência são consequências lógicas.

Completude: o conj. de regras de inferência permite gerar todas as consequências lógicas

Metateoremas

- Teorema da dedução
  Se $\{A_1, A_2, \ldots, A_n\} \models B$, então $A_1 \wedge A_2 \wedge \ldots \wedge A_n \Rightarrow B$, e vice-versa

- Redução ao Absurdo
  Se o conj. de fórmulas Δ é satisfatível e $\Delta \cup \{\neg A\}$ não é satisfatível, então $\Delta \models A$

Scanned by CamScanner

Refutação por Resolução → mecanismo de inferência completo

usa-se a resolução para provar q a negação da consequência lógica é inconsistente c/ a premissa

Passos:
- converter a premissa e a negação da consequência lógica para um conj. de cláusulas
- aplicar a resolução até obter a cláusula vazia

Substituições, Unificação

substituição $s = \{t_1/x_1, \ldots, t_n/x_n\}$, fórmula W
$\Rightarrow$ SUBST(W, s) ou Ws $\Rightarrow$ todas as ocorrências de $x_1, \ldots, x_n$ em W são substituídas por $t_1, \ldots, t_n$

A e B são unificáveis se existe uma substituição s (unificadora) tal que As = Bs

Resolução e Refutação na lógica de 1ª ordem

$\{A \vee B, \neg C \vee D\} \vdash$ SUBST(AVD, g), em q B e C são unificáveis, sendo g a sua substituição unificadora + geral

Resolução com cláusulas de Horn, tem no máx. um literal positivo

ex.: A; $\neg A \vee B$; $\neg A \vee B \vee \neg C$; $\neg A \vee \neg B$

Scanned by CamScanner

Linguagem KIF (Knowledge Interchange Format)

↓

representa o conhecimento trocado entre agentes.

" os modelos internos de cada agente

→ Características principais:

- semântica puramente declarativa
- pode ser tão ou + expressiva q/ a lógica de 1ª ordem
- representação de meta-conhecimento (conhecimento sobre o conhecimento)

Componentes:

- caracteres
- lexemas (palavras, cadeias de caracteres)
- expressões (termos, frases, definições)

⇝ Tende a sobrecarregar os sistemas de geração e de inferência

↳ **solução:** Definir várias dimensões de conformação

→ Perfis de conformação

- lógica: atómica, conjuntiva, positiva, lógica, baseada em regras
- complexidade dos termos: termos simples, termos complexos
- Ordem: proposicional, 1ª ordem, ordem superior
- Quantificação } conforme se usa ou não
- Meta-conhecimento

Engenharia do Conhecimento

processo para criar

Base de conhecimento: conj. de representações de ~~factos e regras~~ frases de funcionamento do mundo.

- estudar o domínio de aplicação (processo de aquisição do conhecimento)
- determinar os objetos, conceitos e relações q/ será necessário representar
- escolher um vocabulário para entidades, funções e relações (ontologia)
- codificar conhecimento genérico sobre o domínio (conj. de axiomas)
- " descrições para problemas concretos, interrogar o sistema e obter respostas
- aprendizagem automática

Scanned by CamScanner

Identificação de objetos, conceitos e relações

exemplos:
- substantivos comuns → conceitos (classes/tipos)
- " próprios → objetos (instâncias)
- verbo "ser" → relação de instanciação (objeto/tipo) ou generalização (subtipo/tipo)
- verbos "ter" e "conter" → relação de composição
-

Ontologia: vocabulário sobre um domínio conjugado de relações hierárquicas como membro e subtipo

Objetivo: captar a essência da organização do conhecimento num domínio

Ontologia geral: aplicável a uma grande variedade de domínios de aplicação

noções:
- categorias / tipos / classes
- medidas numéricas
- objetos compostos
- tempo, espaço e mudança
- eventos e processos
- objetos físicos
- substâncias
- objetos abstratos e crenças

Redes de crença bayesianas

Redes de Bayes

grafo dirigido acíclico → não há ciclo

representam conhecimento impreciso em termos de um conjunto de variáveis aleatórias e respetivas dependências

dependências → probabilidades condicionadas

Scanned by CamScanner

Axiomas das probabilidades

→ Para uma qualquer proposição a, a sua probabilidade é um valor entre 0 e 1:

$$0 \leq P(a) \leq 1$$

→ Proposições necessariamente verdadeiras têm probabilidade 1:

$$P(true) = 1$$

→ Proposições necessariamente falsas têm probabilidade 0:

$$P(false) = 0$$

→ A probabilidade da disjunção é a soma das probabilidades subtraída da probabilidade da intersecção:

$$P(a \vee b) = P(a) + P(b) - P(a \wedge b)$$

Probabilidades condicionadas

$P(a|b)$ ⇒ probabilidade de a proposição a ser verdadeira sabendo q b é verdadeira

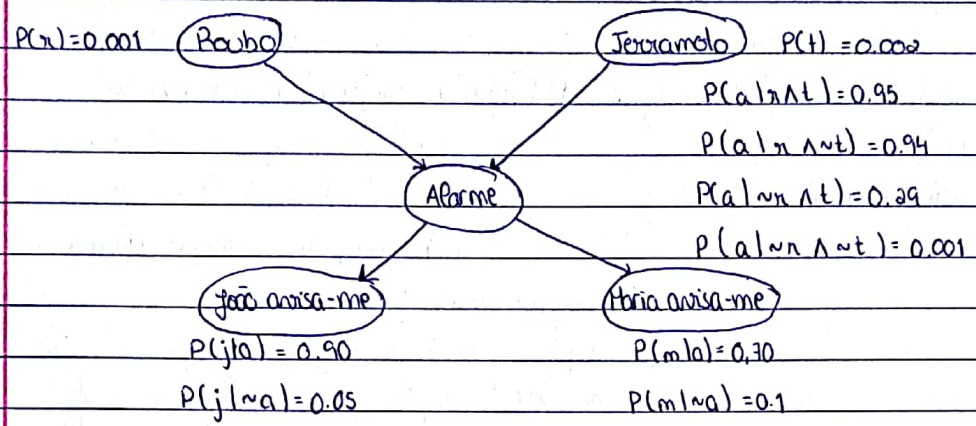
$$P(a|b) = \frac{P(a \wedge b)}{P(b)}$$

Probabilidade conjunta ⇒ probabilidade de ocorrer uma dada combinação de valores de todas as variáveis da rede

$$P(x_1 \wedge \dots \wedge x_n) = \prod_{i=1}^{n} P(x_i | pais(x_i))$$

Scanned by CamScanner

Exemplo Rede bayes ⊕ prob. conj.



⇝ Probabilidade de o alarme tocar e o João e a Maria avisarem num cenário em que não há roubo nem terramoto:

$$
\begin{aligned}
&P(j \wedge m \wedge a \wedge \sim t \wedge \sim r) \\
&= P(j|a) \times P(m|a) \times P(a|\sim r \wedge \sim t) \times P(\sim r) \times P(\sim t) \\
&= 0.90 \times 0.70 \times 0.001 \times 0.999 \times 0.998 \\
&= 0.000628
\end{aligned}
$$

Redes de bayes em Python

- classe. BayesNet()
- classe ProbCond(var, mother_vals, prob) = probabilidade condicionada de uma dada variável ser verdadeira dados os valores (True/False) das variáveis mães

insert : introduzir uma nova probabilidade condicionada na rede

joint_prob: obter a probabilidade conjunta para uma dada conjunção de valores de todas as variáveis da rede

Scanned by CamScanner

Probabilidade individual : probabilidade de um valor específico (verdadeiro/falso) de uma variável

⇓

Soma das probabilidades conjuntas das situações em q essa variável tem esse valor específico

$$P(x_i = v_i) = \sum_{\substack{a_j \in \{v,f\} \\ j=1,\dots,K}} P(x_i \wedge a_1 \wedge \dots \wedge a_K)$$

$C = \{x_1, \dots, x_n\}$ - conj. de variáveis da rede

$x_i \in C$ - qqr variável da rede

$v_i \in \{v, f\}$ - valor de x_i cuja prob. se quer calcular

$\{a_1, \dots, a_K\} \subset C$ - conj. das variáveis da rede q são ascendentes de x_i

Formulação de problemas e pesquisa de soluções

inclui:
- descrição do ponto de partida - o estado inicial
- conj. de transições de estado
- função q diz se um dado estado satisfaz o objetivo
- (opcional) função q avalia o custo de uma solução

processo q vai executando transições de estados até q um estado gerado satisfaça o objetivo

Pesquisa em árvore em Python

SearchDomain() : classe abstrata q formata a estrutura de 1 domínio de aplicação
SearchProblem(domain, initial, goal) : especificação de problemas concretos a resolver
SearchNode(state, parent) : nós da árvore de pesquisa
SearchTree(problem)

Pesquisa em profundidade

Sem repetição de estados : para evitar ciclos infinitos, convém garantir q os estados já visitados no caminho que liga o nó atual à raiz da árvore ñ são novamente gerados

com limite : não são considerados para expansão os nós da árvore cuja profundidade excede um dado limite

com limite crescente :
1) tenta resolver o problema c/ um dado limite N
2) se for encontrada a solução, retorna
3) incrementar N
4) voltar ao passo 1)

Scanned by CamScanner

Avaliação das estratégias de pesquisa

- **Completude**: é capaz de encontrar uma solução quando existe uma solução
- **Complexidade temporal**: quanto tempo demora a encontrar a solução
- **Complexidade espacial**: qt espaço de memória é necessário para encontrar uma solução
- **Optimalidade**: consegue encontrar a melhor solução

Pesquisa A*

→ Escolher o nó em que a função de custo total $f(n) = g(n) + h(n)$ tem o menor valor

- $g(n)$: custo desde o nó inicial até ao nó n
- $h(n)$: custo estimado desde o nó n até à solução [heurística]

2 variantes

→ **custo uniforme**
- $h(n) = 0$
- $f(n) = g(n)$ (custo real)

→ **pesquisa gulosa (greedy)**
- ignora custo acumulado $g(n)$
- $f(n) = h(n)$ (custo estimado)

$h(n)$ é admissível se nunca sobrestima o custo real de chegar a uma solução a partir de n, ou seja, se custo real ≥ custo estimado

Fatores de ramificação

N - nº de nós da árvore de pesquisa no momento em q se encontra a solução
X - nº de nós expandidos
d - comp. do caminho na árvore correspondente à solução

Ramificação média (dificuldade do problema)

$RM = \frac{N-1}{X}$; nº médio de filhos por nó expandido

Fator de ramificação efetivo - nº de filhos por nó, B, árvore d profundidade constante d

$1 + B + B^2 + \dots + B^d = N \approx \frac{B^{d+1} - 1}{B - 1} = N$

Scanned by CamScanner

Pesquisa A*

Vantagens
- completa
- ótima

Desvantagens:
- consumo de memória e tempo com comportamento exponencial
- pode ser preciso algoritmos mais eficientes

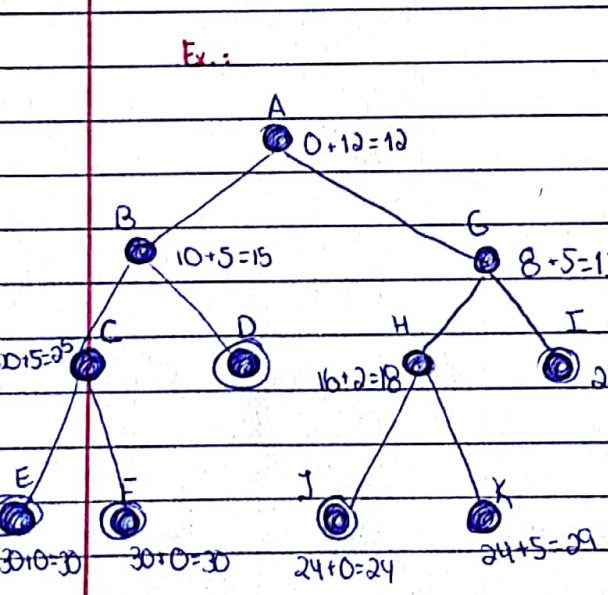
IDA*: A* d aprofundamento iterativo
↳ limitação à profundidade é estabelecida indiretamente através de um limite na função de avaliação $f(n) = g(n) + h(n) \leq f_{max}$

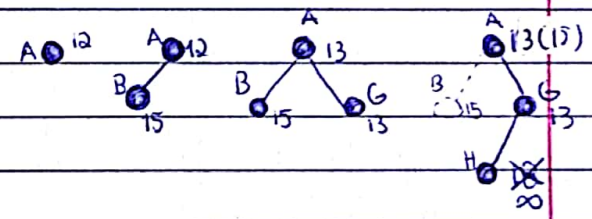
RBFS: Recursive Best-First Search
↳ para cada nó n, o algoritmo não guarda o valor da função de avaliação $f(n)$, mas sim o menor valor $f(x)$, sendo x uma folha descendente do nó n

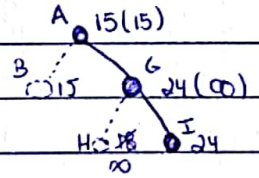
SMA*: A* com memória limitada simplificada (usa a memória disponível)
→ quando a memória chega ao limite, "esquece" (remove) o nó n c/ maior custo $f(n) = g(n) + h(n)$, atualizando em cada um dos nós ascendentes o "custo do melhor nó esquecido"; volta a gerar o nó n quando o custo do melhor nó esquecido registado no antecessor de n for inferior aos custos dos restantes nós.

→ Em cada iteração, é gerado apenas um nó sucessor

Ex.:







Scanned by CamScanner

Problemas de atribuição: atribuir valores a um conj. de variáveis, respeitando um conj. de restrições

Ex.: problema das 8 rainhas; invenção de palavras cruzadas

Técnicas de resolução:
- método construtivo
- " "
- resolução por melhorias sucessivas

combinado c/ propagação de restrições

constrói um grafo de restrições

Algoritmo:

1. Inicialização: o nó inicial da árvore de pesquisa é composto por todas as variáveis e todos os valores possíveis para cada uma delas;
2. Se pelo menos uma variável tem um conj. de valores vazio, falha e retrocede; se não puder retroceder, a pesquisa falha;
3. Se todas as variáveis têm exatamente um valor possível, tem-se uma solução; retornar c/ sucesso;
4. Expansão: Escolher arbitrariamente uma variável V_K e, de entre os valores possíveis, um dado valor X_{Ke} - descartar os restantes valores possíveis dessa variável
5. Propagação de restrições: para cada arco (i,j) no grafo de restrições, remover os valores na variável Vi por forma a que o arco fique consistente.
6. Caso tenha sido preciso remover valores na origem de algum arco, voltar a repetir o passo 5.
7. Voltar ao passo 2

em cada nó do grafo está uma variável

um arco dirigido liga um nó i a um nó j se o valor de j impõe restrições ao valor de i.

Tipos de restrições

Unárias
podem ser satisfeitos através de pré-processamento do domínio de valores da variável - aproveitam-se apenas os valores q satisfazem a restrição

Binárias
é diretamente representada por uma aresta no grafo de restrições

De ordem superior
através da introdução de variáveis auxiliares, uma restrição de ordem superior pode ser transformada num conj. de restrições binárias e/ou unárias

Scanned by CamScanner

Ex.: quebra-cabeças criptoaritmético

$$\begin{array}{r} TWO \\ +\ TWO \\ \hline FOUR \end{array}$$

- Variáveis principais: F, O, R, T, U, W ($\in 0..9$)
- Variáveis internas: x_1 (transporte das unidades para as dezenas) e x_2 (transporte das dezenas para as centenas) ($\in 0..1$)
- Restrições:
  - todas as variáveis são diferentes [restrição sobre 6 variáveis]
  - $2 \times O = R + 10 \times x_1$ [" " 3 "]
  - $2 \times W + x_1 = U + 10 \times x_2$ [" " 4 "]
  - $2 \times T + x_2 = O + 10 \times F$ [" " " "]

Restrições de ordem superior - conversão para restrições binárias

A restrição $2 \times O = R + 10 \times x_1$ pode ser transformada no seguinte conj. de restrições:

~ restrições binárias:
- O = primeiro (Aux)
- R = segundo (Aux)
- x_1 = terceiro (Aux)

~ restrição unária
- 2 × primeiro (Aux) = segundo (Aux) + 10 × terceiro (Aux)

variável auxiliar
cujo domínio é o produto cartesiano
dos domínios de O, R e x_1.

$Aux \in 0..9 \times 0..9 \times 0..1$

Pesquisa por melhorias sucessivas
- Técnicas ⊕ comuns:
- Reparação heurística
- Montanhismo
- Recozimento simulado
- Algoritmos genéticos

Scanned by CamScanner

Montanhismo

→ escolhe-se sempre o sucessor c/ melhor valor da função de avaliação

→ não há retrocesso (backtracking)

→ qd o valor da função no nó atual é superior ao valor da função em qualquer dos seus sucessores, a pesquisa pára.

variante

Recozimento simulado

• podem ser aceites refinamentos que piorem a solução

" deixar espiar lentamente num forno especial "

particularidades:

- sucessor é selecionado aleatoriamente
- qd o valor da função no nó atual é superior ao valor da função no sucessor, o sucessor é aceite c/ uma probabilidade q diminui exponencialmente em função da perda na função de avaliação
- pesquisa termina qd um indicador ("temperatura") chega a zero.

Scanned by CamScanner